JINLING INSTITUTE OF TECHNOLOGY
Department of Software Engineering

USAMIS

University Student Academic Management Information System

Investigation, Design and Implementation Report

A role-based system built with Python, Flask and PostgreSQL

Student	[Your Name]
Student ID	[Student ID]
Course	Management Information Systems
Instructor	[Teacher Name]
Date	[Date of submission]

Abstract

Student records in many universities are still kept in paper files and separate spreadsheets. This causes duplicate data, mistakes, slow reports and no control over who can change a record. This report describes the investigation, design and implementation of USAMIS, a University Student Academic Management Information System.

USAMIS is a web application built with Python and Flask, with PostgreSQL as its database. It has three layers: a web interface, application services and a data layer. Five roles (Admin, Registrar, Lecturer, Finance Officer and Student) use role-based access control, so each person can only do the work of their role. Passwords are stored as bcrypt hashes, all queries are parameterized, and important actions are written to an audit log.

The system manages students, courses, enrollment with duplicate prevention, grades with automatic GPA, fees and management reports. The report presents a context diagram, a data flow diagram, an enrollment flowchart, an architecture diagram and an ER diagram, together with the core code, screenshots and test cases.

Keywords: management information system; student records; Flask; PostgreSQL; role-based access control

摘要

许多高校的学生信息仍然分散保存在纸质档案和不同的电子表格中，容易出现数据重复、录入错误、报表生成慢，以及无法控制谁可以修改记录等问题。本文介绍了“大学学生学业管理信息系统（USAMIS）”的调查、设计与实现过程。

USAMIS 是一个基于 Python Flask 开发的网页应用，使用 PostgreSQL 数据库存储数据，系统分为网页界面、应用服务和数据层三层结构。系统设有管理员、教务员、教师、财务人员和学生五种角色，通过基于角色的访问控制（RBAC）限制每位用户只能执行与其角色相关的操作。用户密码使用 bcrypt 加密保存，所有数据库查询均采用参数化方式，重要操作会记录到审计日志中。

系统实现了学生档案管理、课程管理、选课（防止重复选课）、成绩录入与 GPA 自动计算、学费管理和管理报表等功能。报告给出了系统上下文图、数据流图、选课流程图、系统架构图和实体关系图，并附有核心代码、运行截图和测试用例。

关键词：管理信息系统；学生学业管理；Flask；PostgreSQL；基于角色的访问控制

Table of Contents

1. Introduction....1
1.1 Background....1
1.2 Problem Statement....1
1.3 Aim and Objectives....1
1.4 Scope....1
1.5 Structure of the Report....2
2. Investigation and Requirements....3
2.1 How the Investigation Was Done....3
2.2 Current Process and Its Problems....3
2.3 Users and Roles....3
2.4 Functional Requirements....3
2.5 Non-Functional Requirements....4
3. System Analysis and Design....5
3.1 System Context Diagram....5
3.2 Data Flow Diagram (Level 1)....6
3.3 Student Enrollment Flowchart....7
3.4 System Architecture....8
3.5 Entity Relationship Diagram....9
4. Database Design....10
4.1 Overview....10
4.2 Tables....10
4.3 Key Relationships....10
4.4 Core SQL....10
4.5 Design Decisions....11
5. Implementation....12
5.1 Technology Stack....12
5.2 Project Structure....12
5.3 Database Connection....12
5.4 Authentication....13
5.5 Sessions and Logout....13
5.6 Role-Based Access Control....13
5.7 User Management....14
5.8 Student Registration....15
5.9 Course Management....15
5.10 Enrollment....15
5.11 Grades and GPA....15
5.12 Fees....16
5.13 Management Reports....16
5.14 Audit Logging....16
6. System Screenshots....17
7. Testing....20
7.1 Approach....20
7.2 Test Cases....20

7.3 Summary..... 20
8. Security Features.....20
9. Limitations and Future Work.....21
9.1 Limitations..... 21
9.2 Future Work..... 21
10. Conclusion.....21
References.....23
Appendix A. How to Run the System.....24
Appendix B. Glossary.....25

List of Figures

Figure 3.1: USAMIS system context diagram 5
Figure 3.2: Data flow diagram, level 1 6
Figure 3.3: Student enrollment flowchart 7
Figure 3.4: Proposed system architecture 8
Figure 3.5: Conceptual entity relationship diagram 9
Figure 6.1: Login page 17
Figure 6.2: Admin dashboard 17
Figure 6.3: User and role management 17
Figure 6.4: Student records 18
Figure 6.5: Course enrollment 18
Figure 6.6: Grade entry and GPA 18
Figure 6.7: Fee management 19
Figure 6.8: Management reports 19

1. Introduction

1.1 Background

Every university keeps a lot of information about its students: personal details, course registrations, exam marks, tuition payments and attendance. In many places this information still sits in paper files and in spreadsheets that each office keeps for itself. That works while the school is small. As the number of students grows, it gets slow, and mistakes creep in.

A Management Information System (MIS) brings people, processes and technology together so that raw data becomes information that managers can use (Laudon & Laudon, 2020). For a university this means one shared place where the registrar, lecturers, the finance office and the students work with the same up-to-date records, and each person sees only what they are allowed to see.

This report describes the investigation, design and building of such a system. It is called **USAMIS**, short for University Student Academic Management Information System. It is a web application written in Python with the Flask framework, and it stores its data in a PostgreSQL database.

1.2 Problem Statement

The investigation found five problems that appear again and again in the way student records are handled today:

- Student records are scattered across paper files and separate spreadsheets.
- There is no central login, so staff cannot be identified or limited by role.
- Data is duplicated and often inconsistent, which makes management reports hard to produce.
- Nobody controls who can view or edit a record.
- Fee tracking and result processing are slow and easy to get wrong.

Together these problems cost staff time, reduce trust in the data and make it hard for managers to answer simple questions, such as how many students are enrolled in a department or how many have unpaid fees.

1.3 Aim and Objectives

Aim. To design and build a web-based management information system that keeps student academic records in one place and protects them with role-based access.

Objectives. The system should:

1. keep all student academic records in one database;
2. give every user one login page, using a username and a password;
3. apply role-based access control (RBAC) with clear permissions for each role;
4. manage students, courses, enrollments, grades and fees;
5. prevent duplicate records and unauthorized access;
6. produce management reports from live data;
7. keep an audit log of important activity;
8. offer a simple interface that every role can learn quickly.

1.4 Scope

The project covers the core academic and financial administration of one university. The table below shows what is inside and outside the scope of this version.

In scope	Out of scope (future work)
Login, roles and permissions	Mobile application
Student records and course catalog	Online payment gateway
Enrollment with duplicate prevention	Email and SMS notifications
Grade entry and GPA calculation	QR-code attendance
Fee records and balances	Parent portal
Management reports and audit log	Cloud deployment

1.5 Structure of the Report

Chapter	What it covers
2	How the investigation was done, the current problems, users and requirements
3	System design: context diagram, data flow diagram, flowchart, architecture and ER diagram
4	Database design and the main SQL definitions
5	Implementation with the core code and a short explanation of each part
6	Screenshots of the running system
7	Testing
8	Security features
9	Limitations and future work
10	Conclusion

2. Investigation and Requirements

2.1 How the Investigation Was Done

The investigation had three steps. First, we looked at how a typical university office works today: how a student is registered, how marks reach the registrar, and how fees are recorded. Second, we reviewed the earlier Software Requirements Specification (SRS) and the MIS investigation report written for this project. Third, we grouped what we found by user role, so that the needs of each role were clear before any design work started.

The aim of this step was not to collect statistics. It was to understand the daily work well enough to decide what the system must do and what it can leave out.

2.2 Current Process and Its Problems

Area	How it works today	Problem it causes
Student records	Paper files and spreadsheets kept by different offices	Records differ between offices; searching takes a long time
Access	Shared files, no individual accounts	No way to know who changed what; anyone can edit
Enrollment	Forms filled in by hand and typed in later	Duplicate enrollments and typing mistakes
Grades	Lecturers send marks to the registrar in separate files	Delays, lost files and manual GPA calculation
Fees	Receipts and a separate fee sheet	Balances are hard to check; defaulters are found late
Reports	Built by hand from several files	Slow, and the numbers may already be out of date

2.3 Users and Roles

The system has five roles. Each role has its own responsibilities and sees only the pages it needs. University management does not log in directly; it receives the reports that the system produces.

Role	Main responsibility
Admin	Full control of the system: manages users, roles and all data
Registrar	Manages student records, courses and enrollment
Lecturer	Enters grades, but only for their own courses
Finance Officer	Manages fees, payments and balances
Student	Views own profile, results and fees

2.4 Functional Requirements

ID	Requirement
FR1	The system shall let a user log in with a username and password.

ID	Requirement
FR2	The system shall assign each user exactly one role.
FR3	The system shall check the user's permission before every action.
FR4	The system shall let authorized staff add, view, update and deactivate student records.
FR5	The system shall let the registrar manage the course catalog and assign instructors.
FR6	The system shall enroll a student in a course and refuse a duplicate enrollment.
FR7	The system shall let a lecturer enter grades for their own courses.
FR8	The system shall calculate each student's GPA automatically.
FR9	The system shall record fees due and payments, and show each student's balance.
FR10	The system shall show a dashboard with live totals for each role.
FR11	The system shall produce management reports such as students per department and fee defaulters.
FR12	The system shall write an audit log entry for every important action.

2.5 Non-Functional Requirements

- **Security:** passwords are hashed, queries are parameterized and every action is checked against the user's role.
- **Usability:** the interface is simple, uses clear labels and works on common browsers.
- **Reliability:** data is stored in a relational database with constraints that stop bad data.
- **Performance:** common pages such as lists and the dashboard load in a few seconds.
- **Maintainability:** code is split into small modules so one change does not break everything.
- **Portability:** the system runs on any computer that has Python and PostgreSQL.

3. System Analysis and Design

This chapter shows the system from several angles. The context diagram gives the big picture, the data flow diagram shows how data moves inside, the flowchart explains one important process, the architecture diagram shows how the software is organized, and the ER diagram shows the stored data.

3.1 System Context Diagram

The context diagram treats USAMIS as one block and shows who talks to it. There are six outside parties.

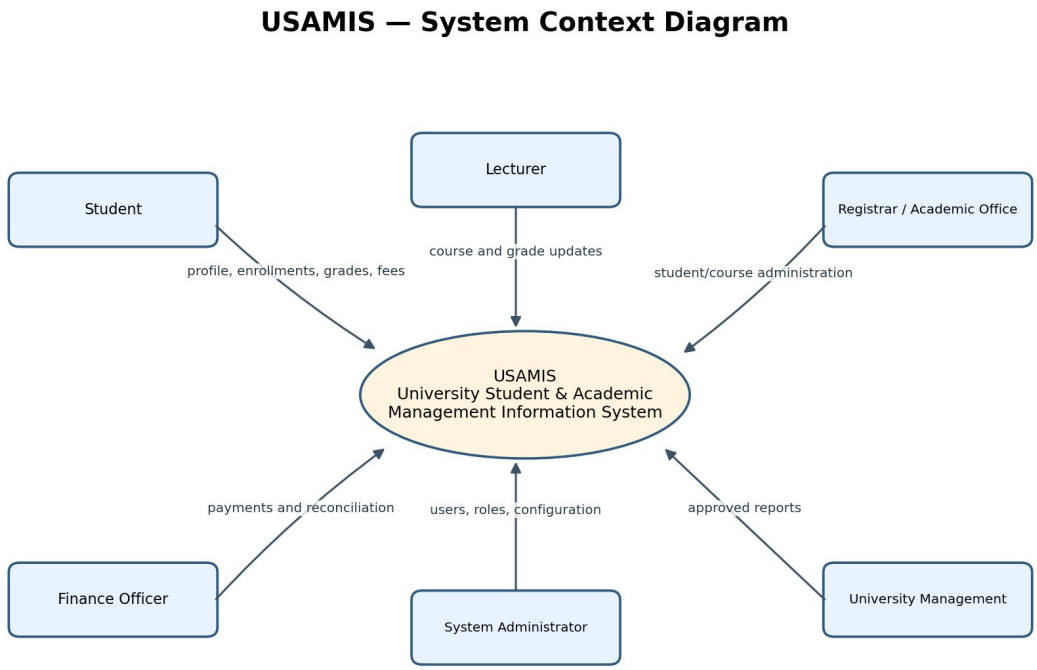


Figure 3.1: USAMIS system context diagram

- **Student:** sends profile, enrollment and fee requests and receives grades and fee status.
- **Lecturer:** sends course and grade updates.
- **Registrar / Academic Office:** sends student and course administration data.
- **Finance Officer:** sends payments and reconciliation data.
- **System Administrator:** manages users, roles and system settings.
- **University Management:** receives approved reports.

3.2 Data Flow Diagram (Level 1)

The level 1 data flow diagram opens the system block and shows three main processes and three data stores.

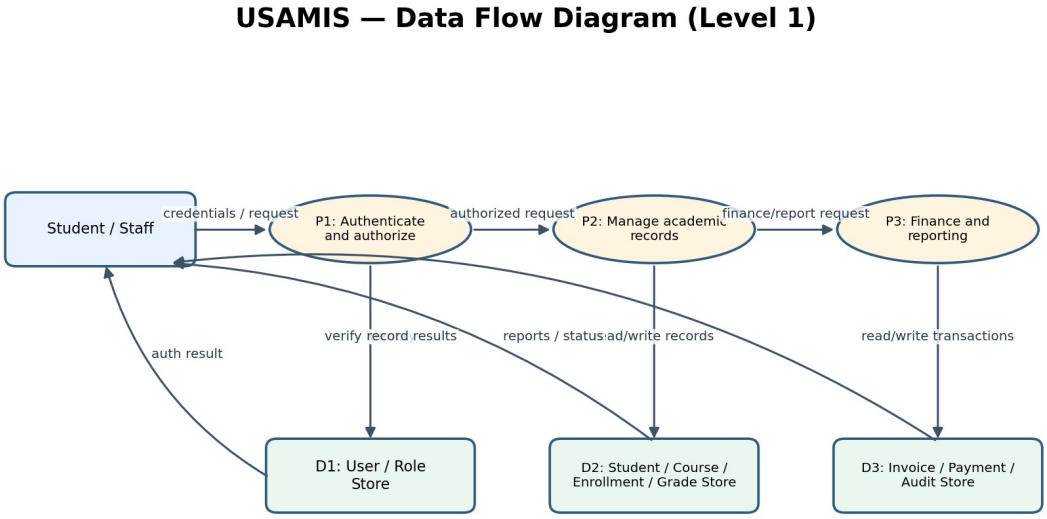


Figure 3.2: Data flow diagram, level 1

- **P1 Authenticate and authorize:** checks the credentials against the user and role store (D1) and returns the result.
- **P2 Manage academic records:** reads and writes students, courses, enrollments and grades (D2).
- **P3 Finance and reporting:** reads and writes invoices, payments and audit records (D3), and prepares reports.
- **D1, D2, D3:** the user/role store, the academic data store and the finance/audit store. In the database these are groups of related tables.

Every request enters through P1. Only an authorized request passes on to P2 or P3. This keeps the security check in one place.

3.3 Student Enrollment Flowchart

Enrollment is the process with the most rules, so it was drawn as a flowchart.

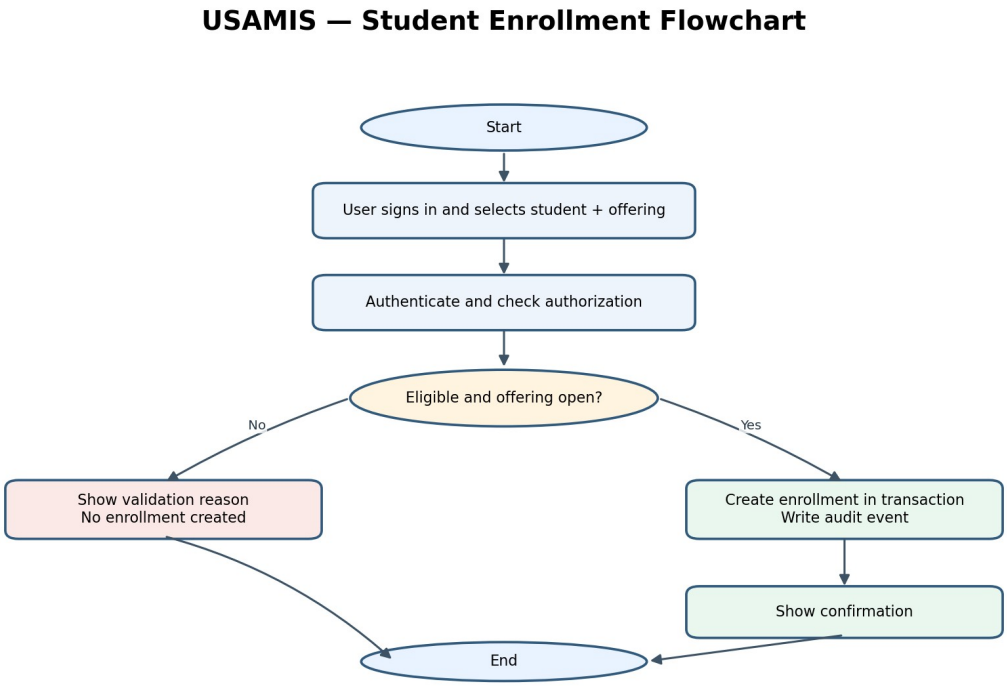


Figure 3.3: Student enrollment flowchart

1. The user signs in and selects a student and a course offering.
2. The system authenticates the user and checks the permission.
3. The system checks that the student is eligible and that the offering is open.
4. If the check fails, a reason is shown and nothing is saved.
5. If it passes, the enrollment is created inside a database transaction and an audit event is written.
6. A confirmation is shown to the user.

Using a transaction means the enrollment and its audit entry are saved together or not at all.

3.4 System Architecture

USAMIS uses a layered design. Each layer talks only to the layer next to it, which makes the code easier to read and to change.

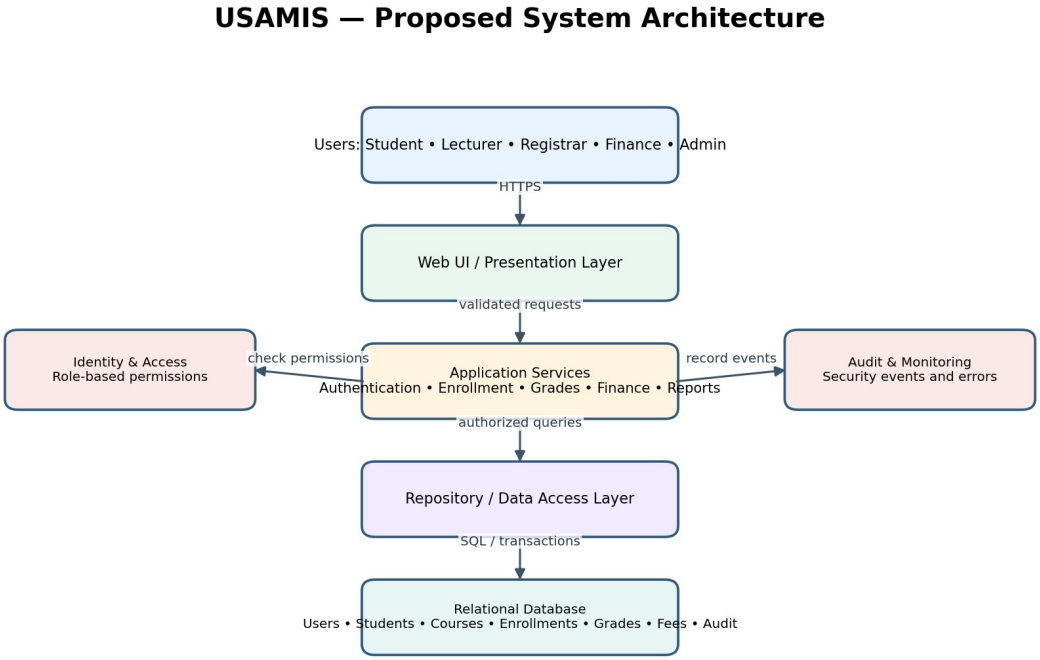


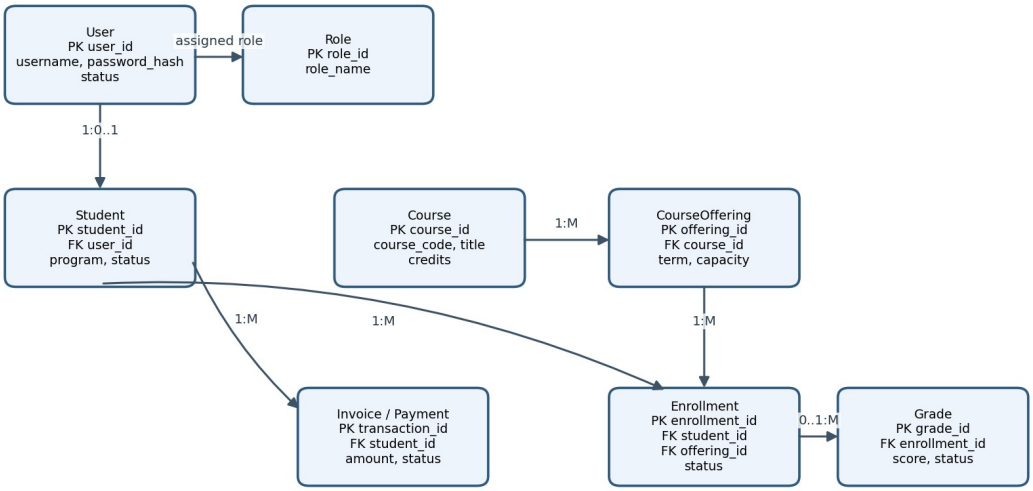
Figure 3.4: Proposed system architecture

- **Web UI (presentation layer):** Flask templates with HTML and Bootstrap. It shows forms, dashboards and reports and sends validated requests down.
- **Application services:** the Flask routes. They handle authentication, enrollment, grades, finance and reports.
- **Repository / data access layer:** the code that runs the SQL, always with authorized queries and transactions.
- **Relational database:** PostgreSQL, holding users, students, courses, enrollments, grades, fees and audit records.
- **Identity and access:** the role-based permission checks used by the services.
- **Audit and monitoring:** records security events and errors.

3.5 Entity Relationship Diagram

The conceptual ER diagram shows the main entities and how they relate.

USAMIS — Conceptual Entity Relationship Diagram



Conceptual model: confirm entities, keys, and cardinalities against the implemented database schema.

Figure 3.5: Conceptual entity relationship diagram

- A **Role** is assigned to many users, and a **User** may have one student profile.
- A **Course** has many **course offerings** (one per term), and each offering has a capacity.
- A **Student** has many **enrollments**, and each enrollment belongs to one offering.
- An **Enrollment** can have a **Grade**.
- A **Student** can have many **invoices / payments**.

The conceptual model is simplified. Chapter 4 gives the physical tables, which add supporting tables such as permissions, departments and the audit log.

4. Database Design

4.1 Overview

The data is stored in PostgreSQL, a relational database that supports constraints and transactions. The tables are grouped into three areas: users and permissions, academic data, and finance and tracking. The design follows the usual rules of normalization so that each fact is stored once.

4.2 Tables

Table	Purpose
users	Login accounts: username, password hash, role and status
roles	The five roles (Admin, Registrar, Lecturer, Finance Officer, Student)
permissions	The list of actions, such as manage_users or enter_grades
role_permissions	Links each role to the permissions it holds
students	Student profile linked to a user account
departments, programs	Academic structure used by students and courses
courses	Course catalog: code, title, credits, instructor
instructors	Lecturers who teach courses
enrollments	Which student takes which course
grades	Score and letter grade for an enrollment
fees	Amount due and amount paid for each student
attendance	Class presence records
audit_log	Who did what, when and to which record

4.3 Key Relationships

- One role has many users.
- One user has one student profile.
- One student has many enrollments.
- One course has many enrollments.
- One enrollment has one grade.
- One student has many fee records.
- One user has many audit log entries.

4.4 Core SQL

The first listing shows the users and roles tables. The second shows how the database itself blocks a duplicate enrollment.

Listing 4.1: Roles and users tables

```
CREATE TABLE roles (
  role_id   SERIAL PRIMARY KEY,
  role_name VARCHAR(30) UNIQUE NOT NULL
);

CREATE TABLE users (
  user_id       SERIAL PRIMARY KEY,
  username      VARCHAR(50) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role_id       INT REFERENCES roles(role_id),
  status        VARCHAR(10) DEFAULT 'active'
);
```

Listing 4.2: Enrollments with a unique pair

```
CREATE TABLE enrollments (
  enrollment_id SERIAL PRIMARY KEY,
  student_id    INT NOT NULL REFERENCES students(student_id),
  course_id     INT NOT NULL REFERENCES courses(course_id),
  status        VARCHAR(15) DEFAULT 'enrolled',
  CONSTRAINT uq_enroll UNIQUE (student_id, course_id)
);
```

4.5 Design Decisions

- **Foreign keys** keep links between tables valid. A grade cannot exist without an enrollment.
- **UNIQUE constraints** stop duplicate usernames and duplicate enrollments, even if the application code has a bug.
- **CHECK constraints** keep values in range, for example course credits between 1 and 6.
- **A status column** lets an account or student be deactivated instead of deleted, so history is kept.
- **Permissions in tables** mean an administrator can change what a role may do without changing the code.

5. Implementation

This chapter shows the core code of the system. Each listing is followed by a short explanation. The code is shortened to the important lines.

5.1 Technology Stack

Part	Tool	Why it was chosen
Language	Python 3	Easy to read, with a large set of libraries
Web framework	Flask	Small and clear; routes, sessions and templates are built in
Database	PostgreSQL	Reliable, supports constraints and transactions
DB driver	psycopg2	Standard way to talk to PostgreSQL from Python
Password hashing	bcrypt	Slow, salted hashing that resists guessing
Front end	HTML and Bootstrap	Quick, responsive pages without extra tools

5.2 Project Structure

Listing 5.1: Folder layout

```
usamis/
  app.py        routes and session setup
  db.py         PostgreSQL connection
  auth.py       bcrypt helpers and the RBAC decorator
  models/       SQL queries for each module
  templates/    Jinja pages with Bootstrap
  static/       CSS and JavaScript
```

Keeping each job in its own file means a developer can find the login code, the database code or the permission code without searching.

5.3 Database Connection

Listing 5.2: Connection function (db.py)

```
import os, psycopg2
from psycopg2.extras import RealDictCursor

def get_conn():
    return psycopg2.connect(
        dbname="usamis",
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASS"],
        host=os.environ.get("DB_HOST", "localhost"),
        cursor_factory=RealDictCursor)
```

All routes open their connection through this one function. The database user and password come from environment variables, so no secret is written in the source code. The dictionary cursor lets the code read a column by name, for example row['username'].

5.4 Authentication

Listing 5.3: Login route (app.py)

```
@app.route("/login", methods=["POST"])
def login():
    u = request.form["username"].strip()
    pw = request.form["password"]
    row = get_user(u)
    if row and check_pw(pw, row["password_hash"]):
        session["user_id"] = row["user_id"]
        session["role"] = row["role_name"]
        audit(row["user_id"], "LOGIN")
        return redirect(url_for("dashboard"))
    flash("Invalid credentials")
```

The route looks up the user, checks the password, and only then stores the user id and role in the session. The same message is shown for a wrong username and a wrong password, so an attacker learns nothing about which one was wrong. A successful login is written to the audit log.

Listing 5.4: Password hashing with bcrypt (auth.py)

```
import bcrypt

def hash_pw(plain):
    h = bcrypt.hashpw(plain.encode(), bcrypt.gensalt())
    return h.decode()

def check_pw(plain, hashed):
    return bcrypt.checkpw(plain.encode(), hashed.encode())
```

bcrypt adds a random salt and is slow on purpose, which makes guessing attacks expensive (Provos & Mazières, 1999). The database stores only the hash. If the database were leaked, the original passwords would not be.

5.5 Sessions and Logout

Listing 5.5: Session setup and logout

```
app.secret_key = os.environ["SECRET_KEY"]

@app.route("/logout")
def logout():
    audit(session.get("user_id"), "LOGOUT")
    session.clear()
    return redirect(url_for("login_page"))
```

Flask signs the session cookie with a secret key so it cannot be changed by the user. Logging out clears the session completely and records the event.

5.6 Role-Based Access Control

RBAC means permissions are given to roles, and users get permissions through their role (Sandhu et al., 1996). The system uses a decorator so that one line above a route protects it.

Listing 5.6: Permission decorator (auth.py)

```
def require_permission(code):
    def wrap(fn):
        @wraps(fn)
        def inner(*a, **kw):
            role = session.get("role")
            if not has_permission(role, code):
                audit(session.get("user_id"), "DENIED")
                abort(403)
            return fn(*a, **kw)
        return inner
    return wrap
```

Listing 5.7: Permission query

```
def has_permission(role, code):
    sql = """SELECT 1 FROM role_permissions rp
      JOIN roles r ON r.role_id = rp.role_id
      JOIN permissions p ON p.perm_id = rp.perm_id
      WHERE r.role_name = %s AND p.code = %s"""
    with get_conn() as c, c.cursor() as cur:
        cur.execute(sql, (role, code))
        return cur.fetchone() is not None
```

The decorator runs before the route. If the role does not hold the permission, the attempt is logged and the user gets an HTTP 403 error. The query uses %s placeholders, so user input is never joined into the SQL text.

5.7 User Management

Listing 5.8: Add a user

```
@app.post("/users")
@require_permission("manage_users")
def add_user():
    f = request.form
    with get_conn() as c, c.cursor() as cur:
        cur.execute(
            "INSERT INTO users(username,password_hash,role_id)"
            " VALUES (%s,%s,%s)",
            (f["username"], hash_pw(f["password"]), f["role_id"]))
    audit(session["user_id"], "ADD_USER")
    return redirect(url_for("users"))
```

Only a role with the manage_users permission can reach this route. The password is hashed before it is saved, and the action is recorded.

5.8 Student Registration

Listing 5.9: Add a student

```
@app.post("/students")
@require_permission("manage_students")
def add_student():
    errors = validate_student(request.form)
    if errors:
        return render_template(
            "student_form.html", errors=errors)
    sid = insert_student(request.form)
    audit(session["user_id"], "ADD_STUDENT", sid)
    return redirect(url_for("students"))
```

Input is checked on the server before anything is saved. If a field is wrong, the form is shown again with the error messages. This protects data quality even if someone bypasses the browser checks.

5.9 Course Management

Listing 5.10: Courses table

```
CREATE TABLE courses (
  course_id     SERIAL PRIMARY KEY,
  course_code   VARCHAR(10) UNIQUE NOT NULL,
  title         VARCHAR(100) NOT NULL,
  credits       INT CHECK (credits BETWEEN 1 AND 6),
  instructor_id INT REFERENCES instructors(instructor_id)
);
```

The course code is unique, credits must be between 1 and 6, and each course points to an instructor. The registrar manages this catalog.

5.10 Enrollment

Listing 5.11: Enroll a student

```
def enroll(student_id, course_id):
    try:
        with get_conn() as c, c.cursor() as cur:
            cur.execute(INSERT_ENROLL,
                        (student_id, course_id))
    except errors.UniqueViolation:
        return False, "Already enrolled"
    audit(session["user_id"], "ENROLL", course_id)
    return True, "Enrolled"
```

The real protection is the UNIQUE constraint from Listing 4.2. If the same pair is inserted twice, PostgreSQL raises an error and the function turns it into a friendly message. Only a successful enrollment reaches the audit call.

5.11 Grades and GPA

Listing 5.12: GPA calculation

```
POINTS = {"A": 4.0, "B": 3.0, "C": 2.0, "D": 1.0, "F": 0.0}

def gpa(rows):
    total = sum(POINTS[r["letter"]] * r["credits"] for r in rows)
    credits = sum(r["credits"] for r in rows)
    return round(total / credits, 2) if credits else 0.0
```

Each grade is turned into points and weighted by the course credits. The last line avoids a division by zero for a student with no graded courses. The result is used on transcripts and in the at-risk report.

5.12 Fees

Listing 5.13: Student balance

```
def balance(student_id):
    sql = """SELECT COALESCE(SUM(amount_due - amount_paid), 0)
             AS bal FROM fees WHERE student_id = %s"""
    with get_conn() as c, c.cursor() as cur:
        cur.execute(sql, (student_id,))
        return cur.fetchone()["bal"]
```

The database does the arithmetic. COALESCE makes a new student show a balance of zero instead of an empty value. A student with a balance above zero after the due date appears in the defaulters list.

5.13 Management Reports

Listing 5.14: Report queries

```
SELECT d.name, COUNT(*) AS students
FROM students s
JOIN departments d ON d.dept_id = s.dept_id
WHERE s.status = 'active'
GROUP BY d.name ORDER BY students DESC;

SELECT ROUND(100 * SUM(amount_paid) / SUM(amount_due), 1)
  AS fee_collection_rate FROM fees;
```

The first query counts active students per department. The second gives the fee collection rate as a percentage. Because the reports are queries on live data, they are always current.

5.14 Audit Logging

Listing 5.15: Audit helper

```
def audit(user_id, action, record_id=None):
    with get_conn() as c, c.cursor() as cur:
        cur.execute(
            "INSERT INTO audit_log(user_id,action,record_id)"
            " VALUES (%s,%s,%s)",
            (user_id, action, record_id))
-- audit_log.created_at DEFAULT now()
```

One small function is called from every module. The database adds the time, so the application cannot set it wrongly. The log covers logins, denied access and changes to users, students and enrollments.

6. System Screenshots

This chapter shows the running system. Each figure was taken after starting the application and opening the page shown.

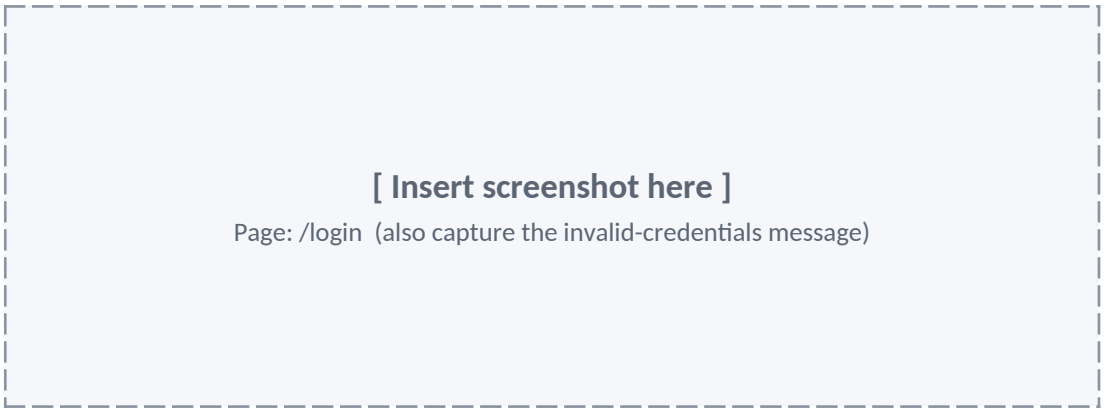


Figure 6.1: Login page

The single login page is used by all five roles. A wrong password shows a short, safe error message.

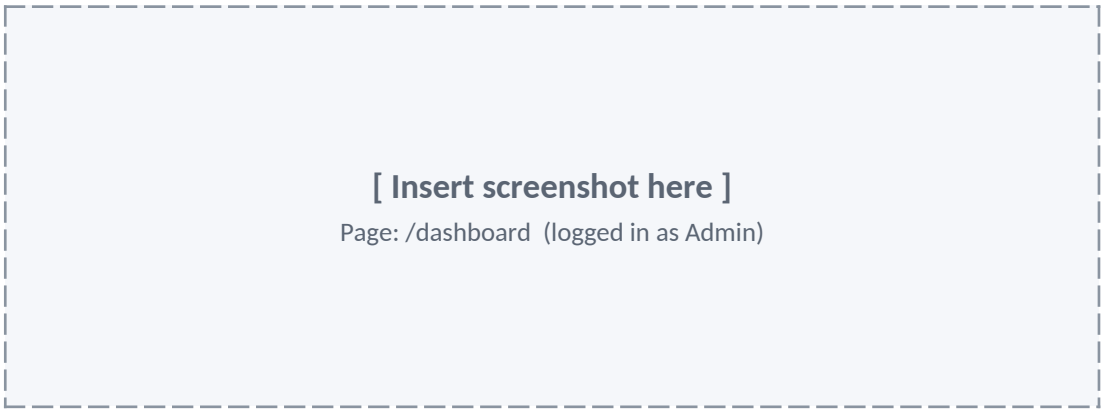


Figure 6.2: Admin dashboard

The dashboard shows live totals such as students, active enrollments, fee collection rate and students at risk.

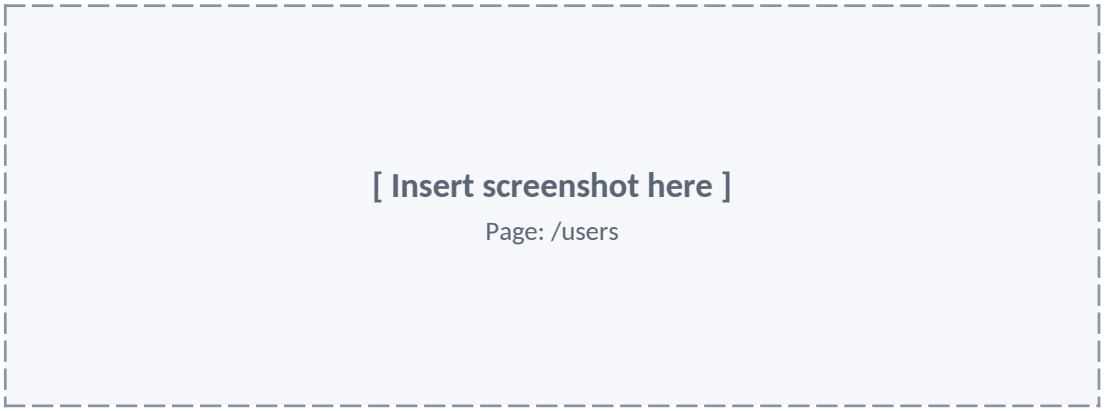


Figure 6.3: User and role management

The administrator adds users, assigns one of the five roles and sets the account status.

[Insert screenshot here]
Page: /students

Figure 6.4: Student records

Authorized staff register, search, update and deactivate students from this page.

[Insert screenshot here]
Page: /enroll (show the 'Already enrolled' message)

Figure 6.5: Course enrollment

The first enrollment succeeds. A second attempt for the same course is refused, which shows duplicate prevention.

[Insert screenshot here]
Page: /grades (logged in as Lecturer)

Figure 6.6: Grade entry and GPA

A lecturer sees only their own courses. The student's GPA updates after the grades are saved.

[Insert screenshot here]
Page: /fees (logged in as Finance Officer)

Figure 6.7: Fee management

The page lists amounts due, amounts paid and balances, and can filter students with unpaid fees.

[Insert screenshot here]
Page: /reports

Figure 6.8: Management reports

Charts show the student distribution and fee status, together with lists such as fee defaulters and low attendance.

7. Testing

7.1 Approach

The system was tested by hand, feature by feature, and by role. For each test we started from a known state, performed the action and compared the result with what the requirement says should happen. Tests for access control were repeated with each of the five roles, because that is where a mistake would be most serious.

7.2 Test Cases

ID	Test	Expected result	Result
T1	Log in with correct details	User is taken to the role dashboard	[Pass / Fail]
T2	Log in with a wrong password	Error shown, no session created	[Pass / Fail]
T3	Admin adds a new user	User appears in the list; action logged	[Pass / Fail]
T4	Lecturer opens the user page	Access denied (403); attempt logged	[Pass / Fail]
T5	Registrar registers a student	Student saved and listed	[Pass / Fail]
T6	Register a student with empty name	Form shows an error; nothing saved	[Pass / Fail]
T7	Enroll a student in a course	Enrollment created; confirmation shown	[Pass / Fail]
T8	Enroll the same student again	Refused with 'Already enrolled'	[Pass / Fail]
T9	Lecturer enters grades for own course	Grades saved; GPA updated	[Pass / Fail]
T10	Lecturer enters grades for another course	Access refused	[Pass / Fail]
T11	Finance records a payment	Balance reduces by the amount paid	[Pass / Fail]
T12	Open the reports page	Figures match the stored data	[Pass / Fail]

7.3 Summary

The test cases cover the three areas that matter most: who can log in, who can do what, and whether the data stays correct. The results column is filled in as each test is run on the final build.

8. Security Features

Security was treated as a basic requirement and not as an extra. The table lists the main measures and the problem each one solves.

Feature	How it is done	Problem it solves
Password hashing	bcrypt with a random salt	Stolen database does not reveal passwords
SQL injection prevention	All queries use parameters (%s)	Malicious input cannot change a query (OWASP, 2023)

Feature	How it is done	Problem it solves
Session management	Signed Flask session, cleared on logout	Session cannot be forged or left open
RBAC	Permission check before every action	Users cannot use features outside their role
Audit logging	User, action, time and record are stored	Misuse can be traced
Input validation	Server-side checks on every form	Bad data does not reach the database
Account status	Accounts are active, inactive or blocked	Leavers and misused accounts can be shut off
Least privilege	Roles get only the permissions they need	A mistake or attack has limited impact

9. Limitations and Future Work

9.1 Limitations

- The system is built for one university and one set of rules.
- Payments are recorded by staff; there is no online payment.
- Reports are basic and are not exported to other formats yet.
- There is no mobile application, so users need a web browser.

9.2 Future Work

Improvement	Benefit
Mobile app for students and staff	Access from a phone
Email and SMS notifications	Reminders for fees and deadlines
Online payment gateway	Students pay without visiting the office
QR-code attendance	Faster and more accurate attendance
AI-based performance prediction	Early warning for students at risk
Parent portal	Families can follow progress
Cloud deployment	Better scale and availability

10. Conclusion

The investigation showed that scattered records, missing access control and manual reports are the main weaknesses of the current way of working. USAMIS answers each of them. All records are kept in one PostgreSQL database, every user signs in through one page, and role-based access control limits each person to the work of their role.

The system covers the full path from registering a student to enrolling in a course, entering grades, tracking fees and producing reports. Duplicate enrollments are stopped by the database itself, passwords are stored as bcrypt hashes and every important action is written to an audit log.

Building the project also taught us how analysis and design connect to code. The diagrams in Chapter 3 were used directly when we wrote the database and the routes. The next step is to add the improvements listed in Chapter 9, starting with notifications and online payment.

References

Laudon, K. C., & Laudon, J. P. (2020). Management Information Systems: Managing the Digital Firm (16th ed.). Pearson.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Sandhu, R., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. IEEE Computer, 29(2), 38–47.

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. Proceedings of the USENIX Annual Technical Conference.

OWASP Foundation. (2023). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org

Pallets Projects. Flask documentation. https://flask.palletsprojects.com

The PostgreSQL Global Development Group. PostgreSQL documentation. https://www.postgresql.org/docs/

Appendix A. How to Run the System

These steps start USAMIS on a local computer. They assume Python 3 and PostgreSQL are already installed.

1. Create a virtual environment and install the libraries.
2. Create the PostgreSQL database and load the schema.
3. Set the environment variables used by the code (database user, password, host and the session secret key).
4. Start the Flask application and open the address shown in the terminal.

Listing A.1: Setup commands

```
python -m venv venv
venv\Scripts\activate          (Linux/Mac: source venv/bin/activate)
pip install flask psycopg2-binary bcrypt

createdb usamis
psql -d usamis -f schema.sql

set DB_USER=postgres
set DB_PASS=your_password
set SECRET_KEY=change_this_value

flask run
# open http://localhost:5000/login
```

Create the first Admin account directly in the database or with a small setup script, then log in and add the other users from the user management page.

Appendix B. Glossary

Short explanations of the technical terms used in this report.

Term	Meaning
MIS	Management Information System: a system that turns data into information managers can use
USAMIS	University Student Academic Management Information System, the system described in this report
RBAC	Role-Based Access Control: permissions are given to roles, and users get them through their role
Flask	A small Python framework for building web applications
PostgreSQL	A relational database that stores data in tables
bcrypt	A password hashing method that adds a random salt and is slow on purpose
Hash	A one-way scrambled form of a password; it cannot be turned back into the password
SQL injection	An attack where harmful text is sent as input to change a database query
Parameterized query	A query where input is passed separately, so it is never treated as SQL code
Transaction	A group of database steps that all succeed or all fail together
Foreign key	A column that points to a row in another table
Audit log	A record of who did what and when
ER diagram	A diagram showing the stored data and how it is related
DFD	Data flow diagram: shows how data moves between processes and data stores
GPA	Grade point average, calculated from grades weighted by course credits

Appendix C. System Screenshots (Full Size)

This appendix reproduces the system screenshots at full size for easy reading. Figure numbers match Chapter 6 of the report.

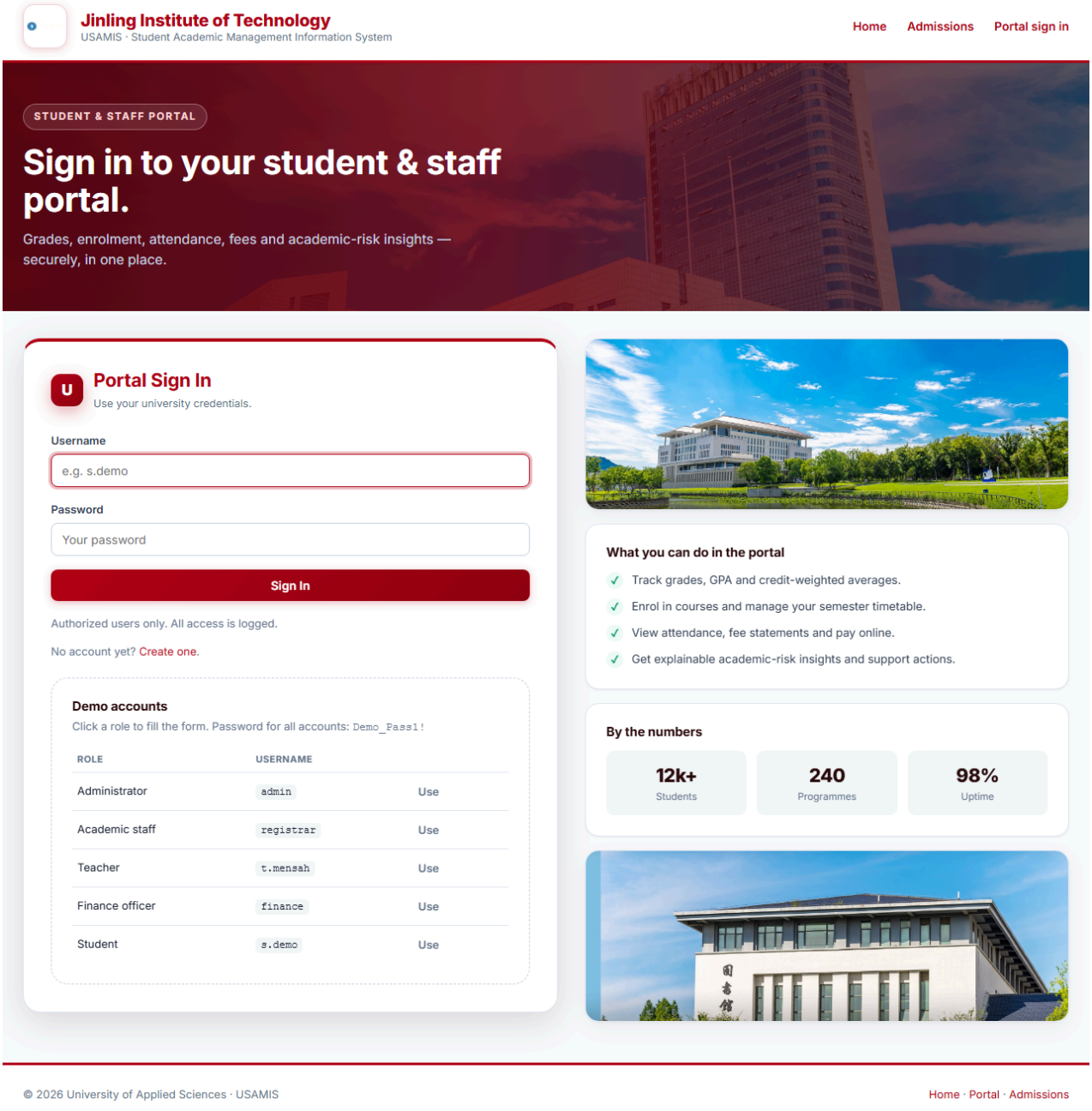


Figure 6.1: Login page

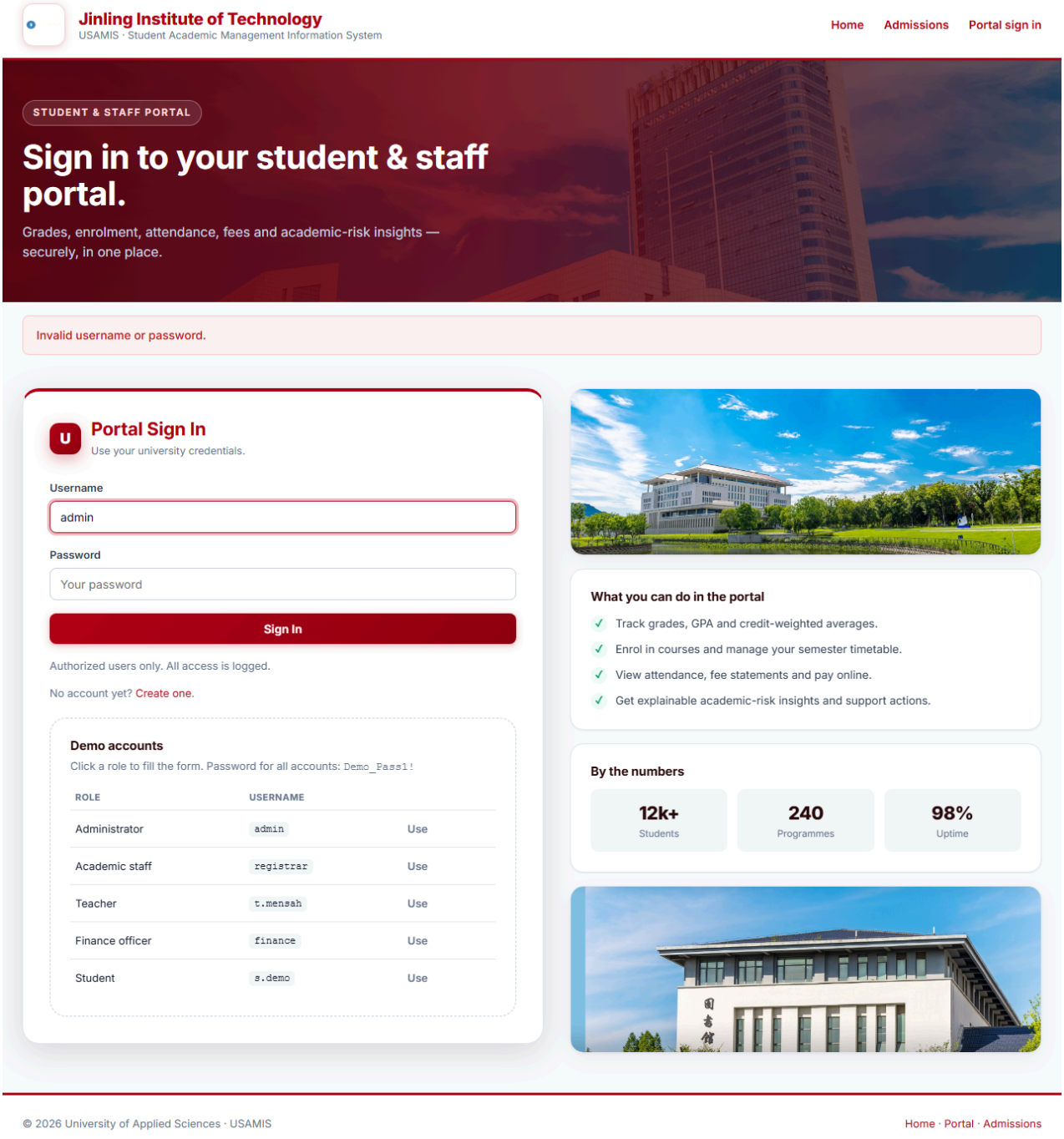


Figure 6.1b: Login page with invalid-credentials message

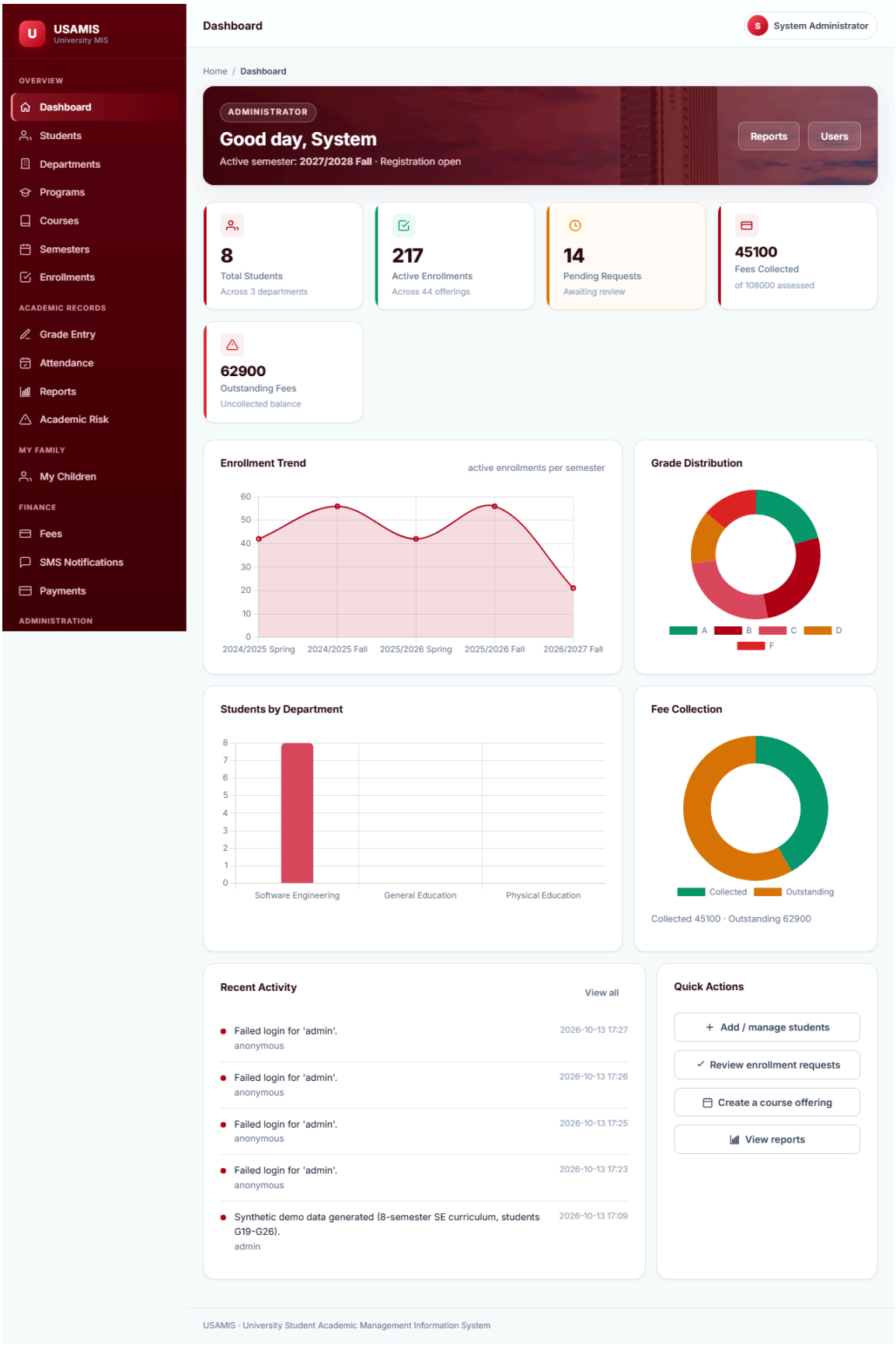


Figure 6.2: Admin dashboard

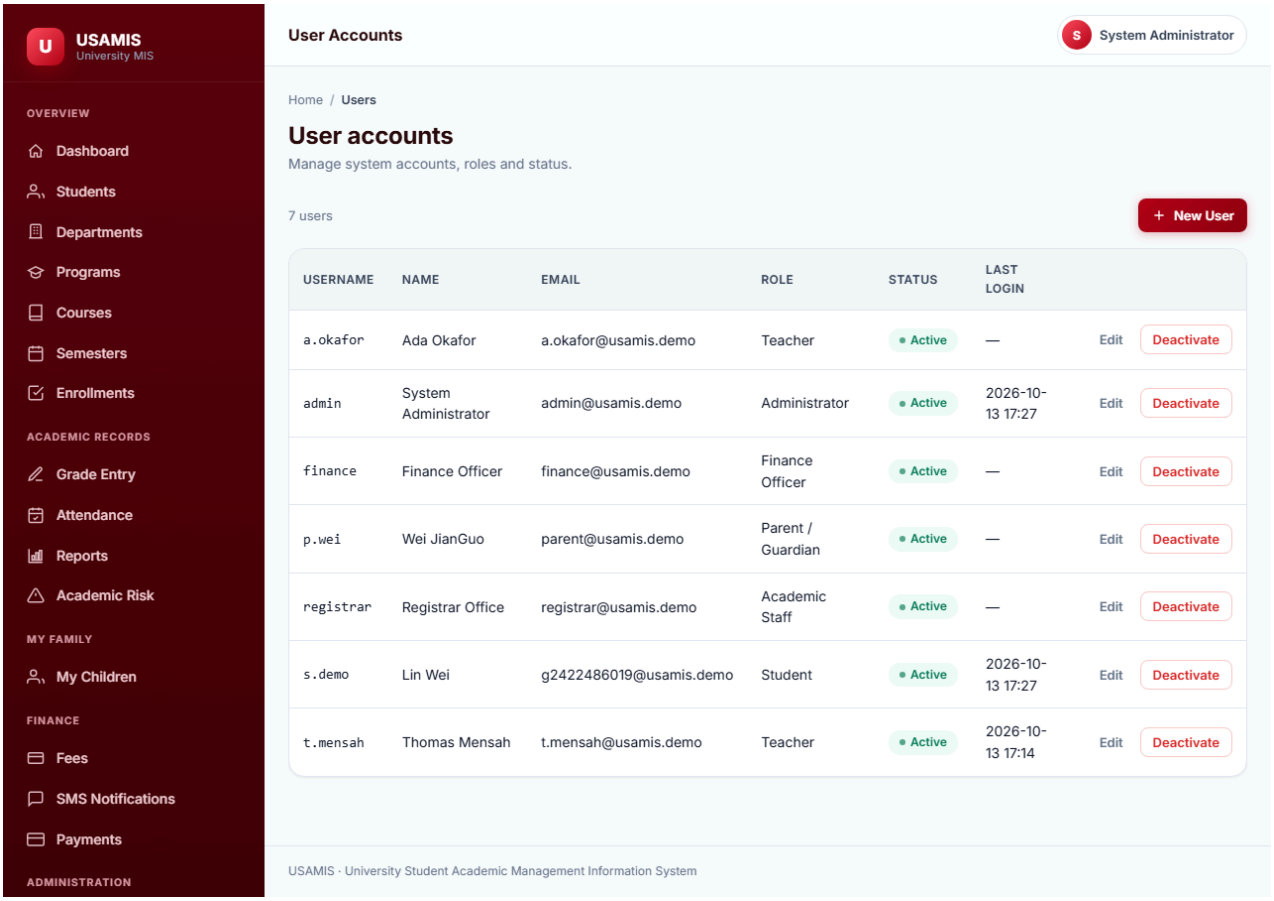


Figure 6.3: User and role management

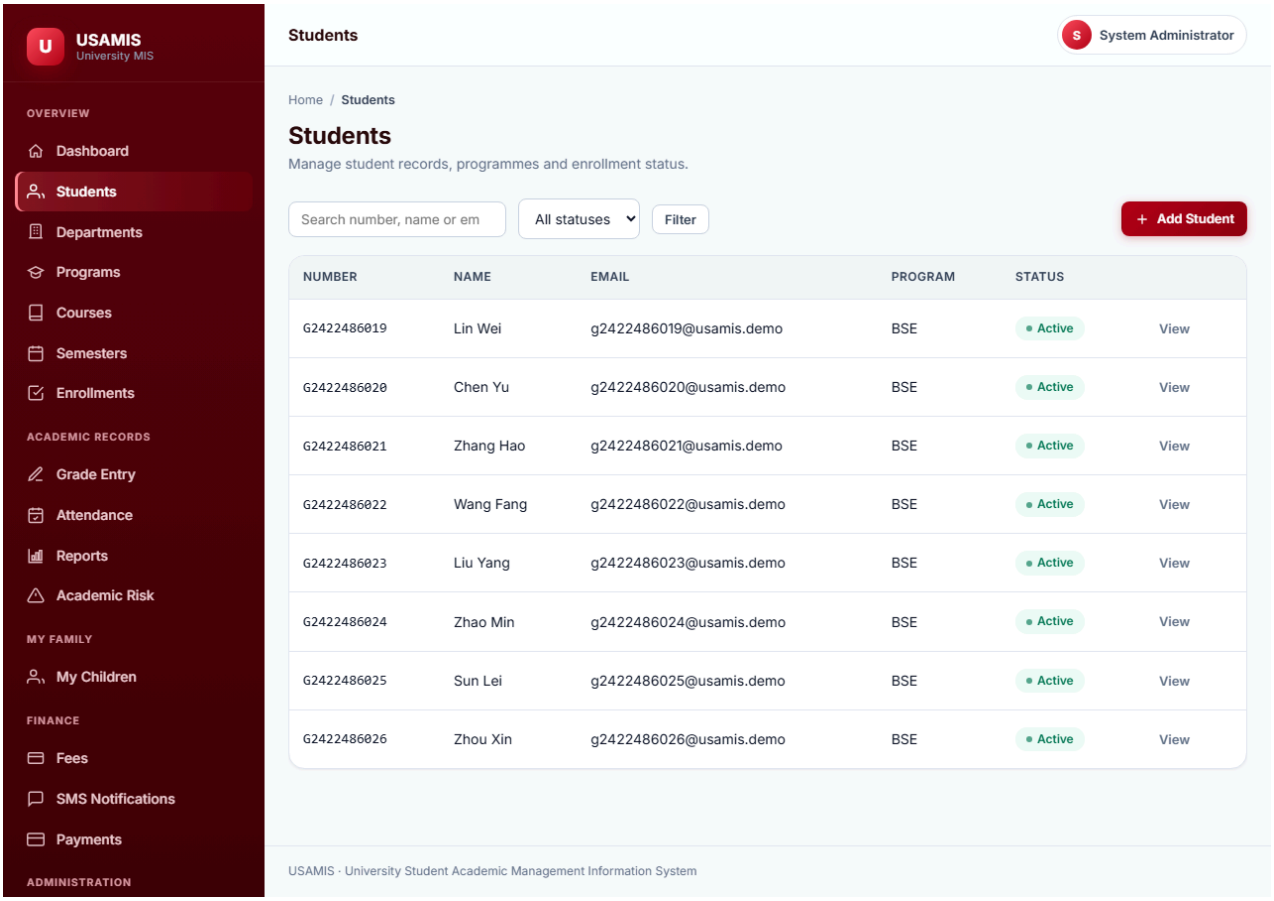


Figure 6.4: Student records

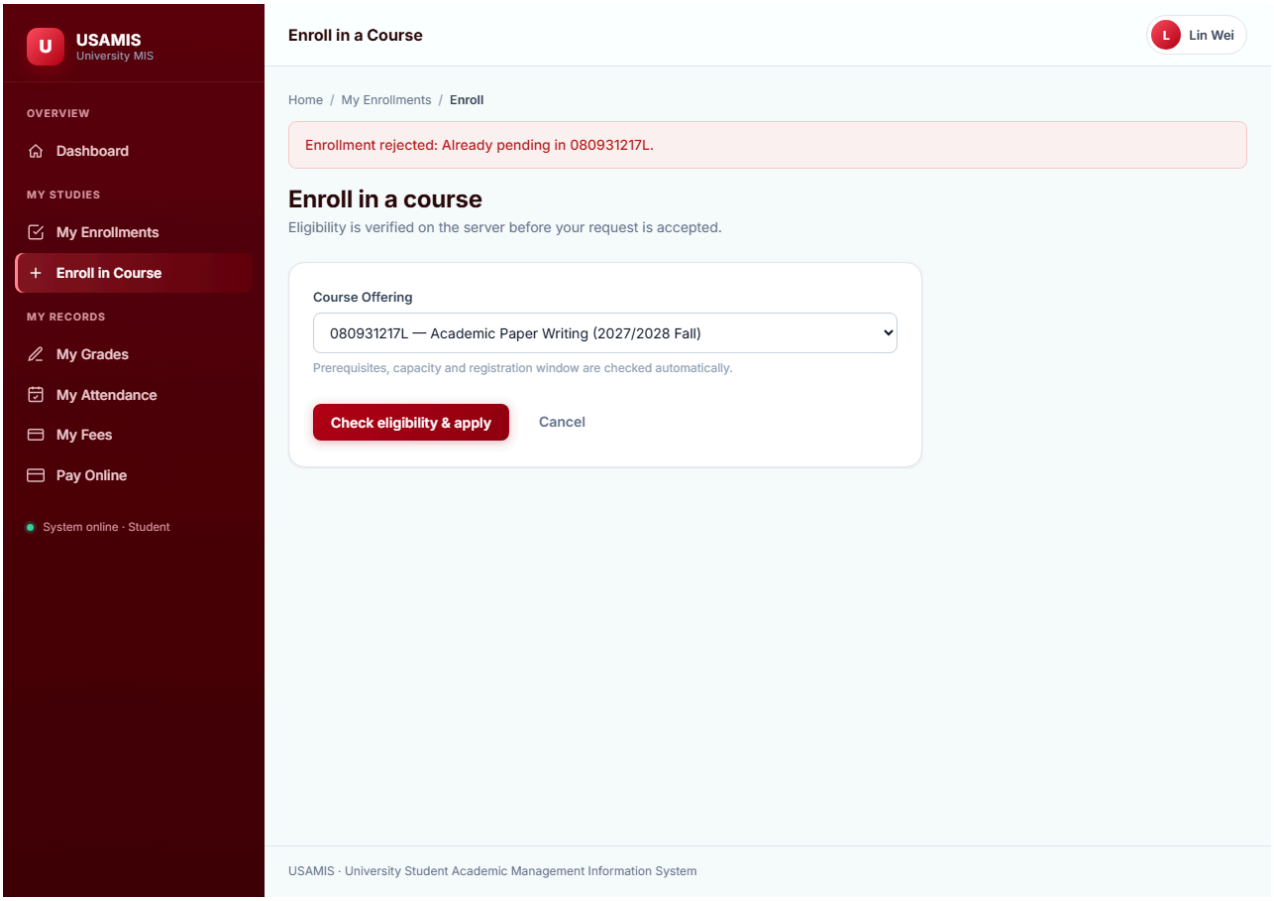


Figure 6.5: Course enrollment

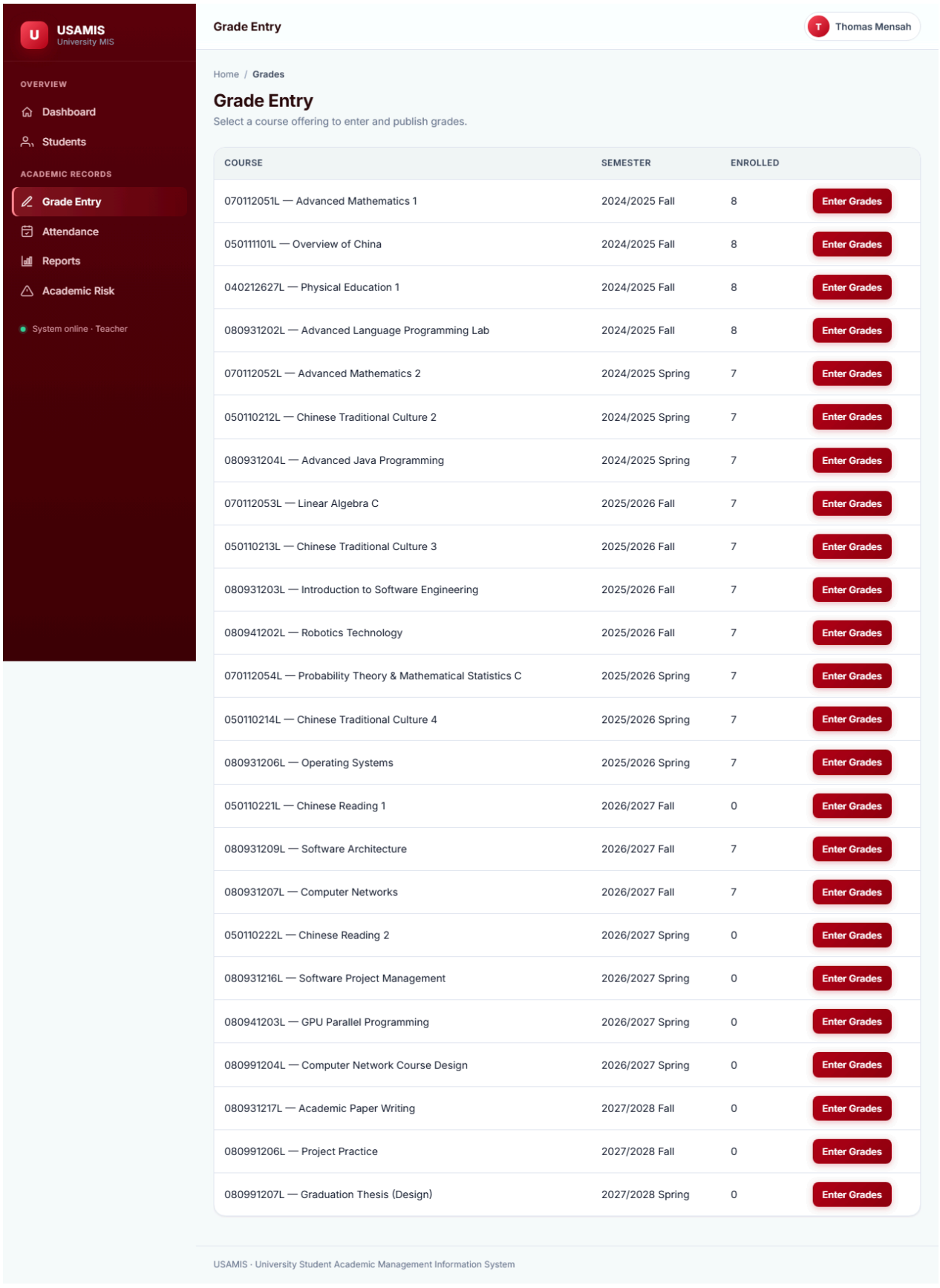


Figure 6.6: Grade entry and GPA

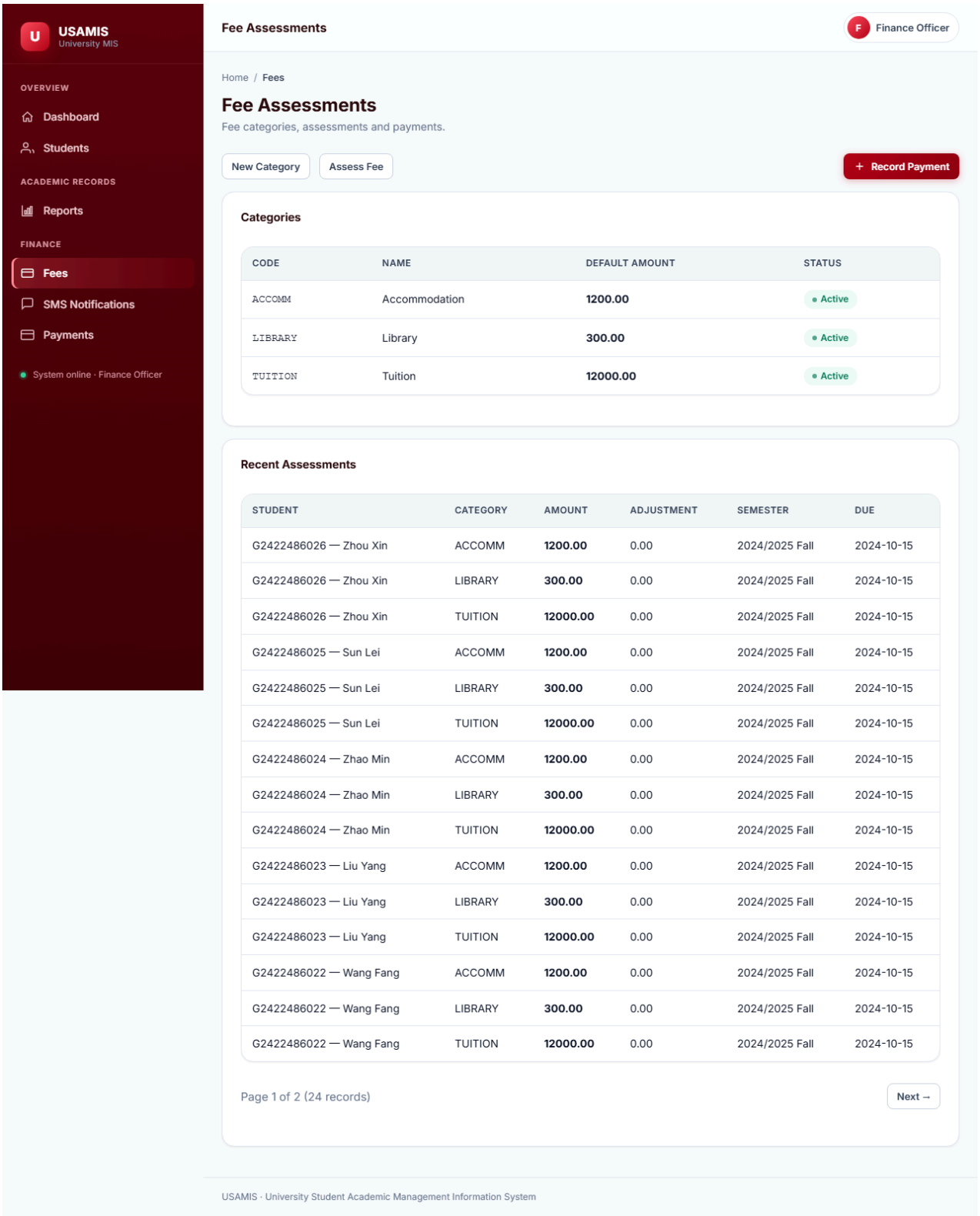


Figure 6.7: Fee management

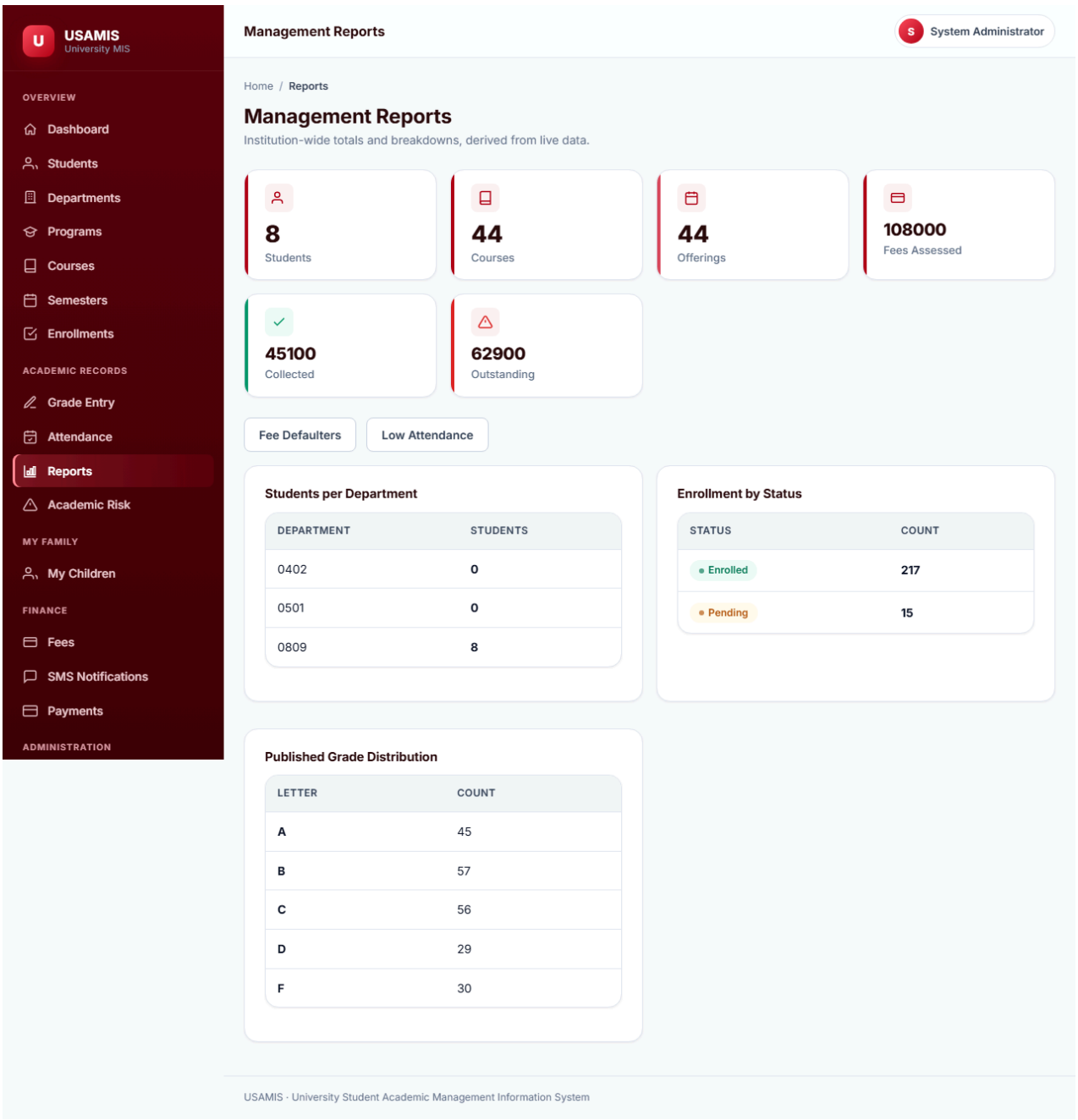


Figure 6.8: Management reports